Agent Hub 演讲稿

Agent Hub 演讲稿

以中文口语为主，保留 Agent、Hub、Workflow、Registry、文件名和字段名等必要英文。对应当前 23 页英文幻灯片，建议按约 30 分钟排练，实际时长以试讲为准。

讲图提示供排练参考，不需要读出来。页面没有笔记浮层，演讲时将本稿单独打开。示例提示词不是原始聊天记录，设计说明与真实运行验证分别表述。

第 1 页｜开场

对应页面：Building Agent Hub with AI

讲图提示：先介绍今天分享的是实现与沟通过程。

大家好，今天想结合 Agent Hub，分享我们怎么和 AI 一起，把一个想法逐渐变成有结构、能继续迭代的项目。

整场只有一个贯穿案例，就是 Agent Hub 本身。我们讲的是怎么与 AI 一起设计和实现这个项目。

我会重点讲三个问题：怎么从需求想到这些 Agent，怎么设计它们之间的 Workflow，以及怎么把讨论变成 AI 能遵循的文件和规则。这里展示的主要是项目设计和定义，真实运行效果，需要另外用调用记录和结果来验证。

第 2 页｜沿着 0918 的路线继续：从可靠任务到 Agent Hub

对应页面：From reliable AI work to Agent Hub

讲图提示：进入页面先讲 0918 区域；按一次翻页显示 This Time，再按一次显示 Today’s Question。逐段讲完后继续下一页。

上次 0918，我们用 AE summary review 讲了怎样让一次 AI 任务更可靠：从 Chat 开始，用 structured prompt 交代目标、输入、规则和交付要求，再通过 context 管理保留当前有效的信息，让结果带着可以核对的依据。左边这部分就是上次的重点。

这次沿着这条线继续往右看。当做法逐渐稳定，我们可以把步骤组织成 Workflow；当需要职责明确的能力时，就要讨论 Agent；而多个能力需要共同完成任务时，就进入 Agent Hub 的协调设计。

这条线是帮助大家定位两次分享的概念路线，不代表每个任务都必须经过所有阶段，也不表示项目已经完成实际运行验证。Prompt、Context 和证据检查，在后面的设计中仍然需要。

所以今天重点回答下面这三个问题：谁负责什么，角色之间交接什么，以及工作应该在哪里停下来。我们会用 Agent Hub 的角色、流程和共同文件，把这些问题逐步说具体。

第 3 页｜整场快照：从 Chat 到 Agent Hub

对应页面：From Chat to Agent Hub: the whole journey at a glance

讲图提示：沿中央箭头从左到右讲六个图标，再指出黄色、蓝色、绿色三个阶段区域。节点下方分别是这一阶段的工作和留下的产物，不需要逐项读完。

先用这张图看整场分享。最开始是 Chat，通过对话探索想法。然后把想法整理成 Structured Prompt，明确目标、背景、输入和约束。

经过讨论确认的规则，可以保存成 Reusable Instructions，比如 README 里的共同约定和各个 Agent 的指令。再用 Workflow 明确步骤、判断和交接，形成职责清楚的 Agent 设计，最后由 Agent Hub 统一入口、协调多个能力。

这不是项目真实创建顺序的还原，也不是说把 Prompt 存下来就自动变成 Agent。真正运行还需要环境、工具、状态处理和验证。当前项目能展示设计和定义如何落实，运行效果需要实际证据。

贯穿今天的设计问题是：用户知道想做什么，却不一定知道选哪个 Agent、怎样组合。接下来我们就以 Agent Hub 为例，重点看角色和流程怎样设计，以及怎样通过文件和 AI 一起把它们做具体。

第 4 页｜为什么要做 Agent Hub：选谁、为什么选、怎样完成

对应页面：Why Agent Hub? From too many options to coordinated work

讲图提示：先呈现 01 · Too many agents，讲清选择困难；按一次翻页后再讲 02 推荐解释和 03 协同执行。候选 A/B/C 是示意。

这一页回答为什么要做 Agent Hub。先看左边：公司 Agent 越来越多，用户知道想完成什么任务，但不一定知道该用哪个。光看名称也很难判断适不适合，自己比较的成本就变高了。这是我们的设计问题假设，不是已经做完用户调研的结论。

中间画的是示意候选清单，A、B、C 和适配等级都不是实际运行结果。它表达我们希望提供的第一步帮助：用户描述目标、材料和限制后，系统推荐合适的 Agent，并解释评分依据、优势和限制。这里的匹配评分表达任务适配程度，不是运行成功率，也不是结果准确率。

右边再往前走一步：推荐以后，还希望能帮用户把工作组织起来。简单任务使用一个 Agent，复杂任务形成多 Agent Workflow。用户确认方案、输入和权限后，再执行并交付可以审核的结果。

所以我们不是只做一个 Agent 名单，而是希望串起发现、选择和授权执行。下一页再看，为了提供这样的体验，Hub、Finder、Builder 和 Executor 的职责应该怎样分。

第 5 页｜和 AI 一起确定主体框架

对应页面：How we shaped the main-agent framework with AI

讲图提示：先指左侧关系图上方的四步过程条，再按右侧气泡讲，最后回到左侧关系图。这里确定谁负责，下一页才讨论角色按什么顺序配合。

有了统一入口、推荐和协助执行的想法以后，我先让 AI 帮忙整理系统，不急着写指令。第一轮说明想实现的体验，AI 再把中间工作拆成理解需求、能力匹配、规划、执行和缺口处理。

然后我继续追问：哪些工作应该由主 Agent 承担，哪些值得独立成一个功能 Agent? 规划与执行不要混在一起，各自不能做什么也要写清楚。

最后得到左边的主体框架：Hub 负责统一交互和路由，Finder 匹配，Builder 规划，Executor 在确认后执行，Creator 处理能力缺口及复用方案。

这一页的重点是想法怎样变成职责划分。下一页先用泳道图核对交接，再推导初步 Workflow，并从用户角度追问补分支。气泡是依据现有项目整理的示例，不是原始对话记录。

第 6 页｜用泳道图和 AI 核对职责与交接

对应页面：From role design to runtime handoffs: who does what?

讲图提示：进入页面先看黄色框里的 Owner、Handoff、Boundary 三个问题；按一次翻页，再沿出现的泳道图讲责任和交接。

角色框架确定后，先看黄色框：我们请 AI 把每一步的负责人、交接内容和暂停边界说清楚。按一次翻页，泳道图出现，再沿着图核对每一步谁负责、交出什么，以及谁接手。

先看 **Owner**。Finder 负责匹配和推荐，Builder 负责规划，Executor 负责执行已经确认的任务。这样可以检查有没有两个角色都在做同一件事，或者某一步没有人负责。

再看 **Handoff**，我会沿着图讲三次交接：

- **Finder → Hub**：交回候选 Agent 和适配理由，让 Hub 向用户解释选择。
- **Builder → Hub**：交回方案，写清楚步骤、负责人、输入、输出和依赖，供 Hub 展示和确认。
- **Hub → Executor**：交付已经确认的方案与执行范围，让 Executor 按这个范围调度，而不是重新规划。

最后看 **Boundary**。比如需要的材料还没提供，就先暂停并返回缺失字段，由 Hub 获取输入；方案被拒绝，就回到修改或澄清，不能直接进入执行。

所以我和 AI 沟通时，不只问“几个 Agent 怎么配合”，而是逐步问：谁负责、交出什么、谁接手、什么时候停。泳道图帮助我们把这些约定摆在同一张图里核对。这里展示的是典型路径，完整判断与返回分支会在后面的完整流程页展开。

第 7 页｜有了角色，再推导初步 Workflow

对应页面：From roles to an initial workflow

讲图提示：左边是初步主线，右边是怎样要求 AI 把角色连起来。

角色确定后，我会请 AI 先画从请求到结果的主线，明确角色怎样交接。先得到理解、匹配或核验、必要时规划、确认、执行、审核这条路径。

但这只是初步主线，还没有从用户各种问法检查完整性。接下来我会让 AI 从用户角度考虑：只是浏览怎么办、已经指定 Agent 怎么办、任务需要多个角色怎么办。接下来三页就是通过这些追问补分支。

第 8 页｜有了角色，再推导初步 Workflow

对应页面：From roles to an initial workflow

讲图提示：左侧流程图与上一页保持同一位置。按顺序讲 Hub、Finder、Builder、Executor，右侧对应颜色的类比说明会同步逐行出现。

角色确定后，我会请 AI 先画从请求到结果的主线，明确角色怎样交接。先得到理解、匹配或核验、必要时规划、确认、执行、审核这条路径。

主线框旁直接标出每一步的负责人：Hub 理解需求，Finder 查找或核验 Agent，Builder 在需要时安排 Workflow，Executor 执行确认后的任务。右侧每一行用不同颜色对应左边的角色，按讲解顺序同步出现。类比仍沿用前面的术语：Hub 像 programming lead，接收请求、协调角色并把方案带回确认；Finder 像 macro

library search，说明匹配理由和限制、指出缺少的输入，但不执行；Builder 像 programming planner，在多步骤任务中安排步骤、输入、输出和依赖，简单任务可以跳过规划；Executor 像 batch-job runner，只运行确认过的步骤，并报告完成、失败或跳过，失败时不会自动重跑整个流程。

这只是初步主线，还没有按用户的不同问法检查完整性。接下来会从用户角度追问：只是浏览怎么办、已经指定 Agent 怎么办、任务需要多个角色怎么办，再逐步补上分支。

第 9 页｜判断用户意图

对应页面：User perspective 1: “I am only browsing.”

讲图提示：对比图中左右两条路径。

第一段先判断用户想得到什么。“有哪些 Agent”和“帮我比较两份文件”，虽然都在和 Hub 对话，但期待的结果不同。

前者是了解能力，返回概览或推荐就可以，后者是执行任务，需要检查目标和材料。如果目标本身不清楚，就先问必要的问题，如果目标明确、只是文件还没给，可以先讨论能力，但执行要等待输入。

让 AI 写流程时，需要同时写清楚下一步是什么，以及应该在哪里停下来。

右侧对话可以这样讲：先问“用户只是查询有哪些 Agent，是不是也需要执行”，AI 建议把浏览和执行分开。再追问“目标明确，但还没提供材料怎么办”，最后把规则明确为可以讨论合适能力，但必要输入补齐前不能执行。这里是根据项目规则整理的示例对话，不是原始聊天记录。

这一页承接初步主线，从用户的不同情况出发，通过追问补齐判断分支，而不是事后只解释现成规则。

第 10 页｜选择与核验 Agent

对应页面：User perspective 2: “I already chose an agent.”

讲图提示：先指左侧过程条：初步主线、用户指定 Agent、核验或替代推荐，再讲右侧追问。

第二段是怎么找到合适的 Agent。用户没有指定时，交给 Finder 推荐，用户已经指定时，先检查它是否存在、适不适合这个任务，以及实际能不能运行。

指定有效，可以跳过重新推荐，指定不合适，就说明原因，再寻找替代。这里还要区分 Active 和 runtime readiness。Active 说明定义已完成、可以被发现，不能直接证明工具连接和运行已经通过验证。

一个看起来简单的用户请求，往往包含了好几个需要提前设计的判断。

右侧可以按对话讲：先问“指定 Agent 后还要重新推荐吗”，再追问“选错了怎么办、选择是否等于授权执行”。从这两个追问，推导出核验、替代推荐和独立执行确认三个规则，再对应到左侧分支。示例是重构对话，不是历史原文。

这一页承接初步主线，从用户的不同情况出发，通过追问补齐判断分支，而不是事后只解释现成规则。

第 11 页｜复杂请求怎样决定功能职责

对应页面：User perspective 3: “My task needs several agents.”

讲图提示：先指左侧过程条：初步主线、多 Agent 请求、明确规划与交接，再讲右侧对话。

回到 Agent Hub 的实现：如果用户的一个请求需要多个 Agent，系统应该由谁来拆步骤、谁来执行？这正是我们需要与 AI 讨论的设计问题。

这里把规划交给 Builder，让它返回每一步的输入、输出和 dependencies，执行则交给 Executor，接收已经确认的蓝图。Hub 负责向用户解释计划并确认范围。

可以给 AI 这样的要求：不要把“规划”和“执行”写成同一个模糊职责，明确两个角色通过什么字段交接、什么时候暂停。这样 Workflow 讨论就会落实到功能指令。

右侧先问“多个 Agent 怎样配合”，让 AI 提出按步骤明确负责人、输入输出和依赖。接着追问“谁规划、谁执行、怎样交接”，把 Builder、Hub 和 Executor 的边界说清楚。绿色部分是最终规则：Builder 输出蓝图，Hub 展示并确认，Executor 接收确认后的计划再执行。这个对话是重构示例。

这一页承接初步主线，从用户的不同情况出发，通过追问补齐判断分支，而不是事后只解释现成规则。

第 12 页｜向 AI 追问：下一步走哪里

对应页面：Ask AI to connect the main path, branches, and return paths

前面已经有主线，也补了不同用户情况，现在请 AI 把它们连成一张完整图。重点不是再介绍系统功能，而是展示追问方法：缺少输入以后到哪里、方案被拒绝如何返回修改、执行失败以后怎样处理。要求每条分支有返回、等待或停止位置，发现连接缺失先指出，不擅自编决定。左侧是真实项目完整图，右侧是说明这种沟通方式的整理对话。

讲图提示：左侧展示完整流程图，先说明整体方向，不逐字读缩略图。点击左图打开完整图，用 + 放大，再沿正常路径和返回分支滚动讲解；关闭后回到主画面。约 2-3 分钟。

先沿正常路径走一遍：用户提出请求，Hub 理解目标并检查材料；需要时交 Finder 推荐，指定 Agent 时先核验；多阶段任务再交 Builder 形成 Blueprint；Hub 展示方案并确认范围，Executor 才调度业务 Agent，最后回传结果供审核。

再用三个追问检查这条主线。第一，“Agent 已经选好，但文件下周才给”，应该停在输入检查，返回缺失材料，由 Hub 获取输入；选择 Agent 不等于已经具备执行条件。第二，“用户不接受计划”，应该返回修改和澄清，把变更后的计划再展示，而不是跳过确认继续执行。第三，“某一步执行失败”，先返回失败步骤、原因和已完成状态，等待判断下一步；不能默认从头重跑，也不能把跳过当成成功。

还有一条很短的路径：用户只是问有哪些 Agent，返回概览或候选就结束，不必走规划和执行。流程完整不意味着每次都走完所有节点，而是每一种情况都有明确的下一步、等待或停止位置。

我会把这些具体问题交给 AI，让它沿箭头检查有没有断掉的返回路径。流程图是我们检查设计的工具，这个讲解属于规则走查，并不是一段已经实际运行成功的记录。

第 13 页｜用 AI 整理零散想法，再按约定逐步生成

对应页面：Use AI twice: organize our ideas, then work from the agreed files

讲图提示：先指左边三个散落便签，再指中间 AI 整理图标与 Human review，接着看三份文件，最后沿右侧三个步骤向下讲。底部虚线表示审核后回到共同参照做修正。

其实整个做法可以用这一页概括。我们一开始可能只有零散想法：角色怎么分、文字写成什么风格、README 要有哪些 section，以及后面要按哪些步骤做。

第一轮和 AI 配合，是让它帮忙归类、总结，把缺失和冲突指出来。已经确认的决定和没确定的问题要分开，由我们审核，不让 AI 自己替我们补决定。

确认后，再把这些内容写成持续可用的文件和规则。比如写作风格、格式和长度写进相关规范，README 保存目标、职责、流程和决策，step-by-step 的安排保存到 Workflow 和 TODO。

第二轮再让 AI 做事时，就明确要求它先读取这些文件，按当前步骤生成或修改内容，然后报告检查和未完成项。这样每次不只是重新说一个笼统目标，而是有共同参照和步骤可以核对。

底部还要有反馈循环。如果结果不符合意图，就判断是生成没遵守规则，还是规则本身不够清楚，再修正并写回文件，继续下一步。这里说的生成文件和后续 Agent 的真实调用要分开，运行仍要看工具、权限、授权和验证证据。

接下来就用项目里已有的文件，展示这些共同参照具体长什么样。

第 14 页｜把已有的总结文件整理成可复用框架

对应页面：Four reusable files, four clear jobs

讲图提示：先从左到右讲四类文件的用途，再看 Registry 卡片内的详细说明，依次介绍四类字段和可检索与已验证的区别。

04 部分先看看项目里已经有哪些可重复使用的总结框架。Root README 保存整体约定，业务目录 README 保存业务 Agent 的格式与边界，Registry 统一能力元数据，TODO 保存进度和验证任务。

Registry 像 macro library 的 catalog。Registry 卡片内列出每条记录的状态、能力、必需输入和适用范围，例如是否能核对 spec 与 define.xml、是否只支持 ADaM 而不支持 SDTM。列在 catalog 里只说明可以被找到，不等于已经在某个 study 验证过，也不代表现在就能直接运行。

这些不是凭空加的一套标准，而是从项目已有文件中提炼出来的组织方式。以后交给 AI 新任务时，可以明确指定它应该遵循哪一类文档，而不必把所有要求重新描述一遍。比如新增业务 Agent，要按业务目录的 Name、Description、Instruction 组织，同时同步 Registry。

第 15 页｜对话、固定框架与实际 README

对应页面：Ask AI to summarize our agreement, then inspect the README

讲图提示：从左到右，先看请求，再看框架，最后滚动真实文档。

我们已经讨论了角色、流程和边界，现在让 AI summarize 这些共识，而不是重新设计。左侧对话明确要求保存已接受的决定，并把未定问题列出来。中间是 README 的固定框架，用箭头表示各个 section。右边展示实际的 agent hub / README.md，可以滚动检查这些共识怎样落在文档里。

这里体现的分工是：我们决定内容与结构，AI 帮忙归纳和保存，我们再核对真实文件。对话是示例表达，文档来自当前项目。

第 16 页｜用业务 Agent README 规定格式，再看真实定义

对应页面：Use the business-agent README as a generation standard

左侧是 business-subagents/README.md 的固定要求：Name、Description、Instruction，以及输入缺失、结构化返回和 Registry 一致性。右侧拿真实的演讲排练反馈助手 TXT 做例子，可以滚动查看职责、输入、边界和返回格式。重点是共享标准怎样约束不同业务定义，而不是临时给每个 Agent 发明格式。右侧文件是当前真实产物，示例请求不是原始聊天记录。

第 17 页｜按 TODO 一步一步生成

对应页面：Build one agreed step at a time, then update TODO

讲图提示：先指用户要求，再沿四步讲如何控制本轮范围。

README 和格式明确以后，怎样让 AI 把项目继续做出来？这里要区分两种流程：前面是 Hub 将来处理用户任务的 Workflow，这里是我们借助 AI 构建项目的实施步骤。

我可以先让 AI 读 README 和 TODO，提出顺序，不要一次生成全部内容。本轮只完善 Hub 和 Finder，生成后核对职责、格式与边界，再更新已完成、待定和下一步。

这样 TODO 不只是待办列表，也是控制实现节奏的入口。下一轮从保存的结果继续。这里是示例请求，不声称这段话就是原始历史记录。

第 18 页｜先恢复上下文，再继续工作

对应页面：Resume without losing context or redesigning agreed work

讲图提示：先指左侧三个问题，再沿右侧顶部四步过程条说明继续工作的顺序，接着指参考资料栏和对话，讲恢复上下文与限定修改范围。

这页放在规则检查之前。多轮沟通有三个问题：之前的决定丢了、已经定好的架构又重做、小修改扩散成大改。继续前明确引用当前 README、相关流程图、TXT 和 TODO，让 AI 先总结共识与未完成项，再定位受影响分支和文件。资料冲突先指出，无关部分保持不变。这里不假设 AI 自动记得所有历史。

话术补充：先要求 AI 读取当前文件，归纳角色、规则、已完成和待定事项；再定位这次请求影响的分支和文件，解释建议修改。发现资料冲突或缺少决定时先指出。修改后汇报改了哪些文件、检查了什么、还有哪些问题，改变已接受的职责或规则前先询问。

第 19 页｜用一个具体请求走查修改后的规则

讲图提示：上下四列一一对应，从左到右讲请求、分支、文件核对与定点修正。

对应页面：Check a revision by walking one concrete request

恢复上下文并生成修改后，再沿上面的四步检查。拿“用这个 Agent，文件晚点给”为例，选择 Agent 不等于输入已经齐全。先验证能力，再检查材料，缺少时返回最少缺失字段，由 Hub 获取输入。对照 README、Hub 和相关 TXT，找出不一致的条款，再给 AI 定点反馈。这是设计走查，不是已执行的运行测试。

第 20 页｜改完文件以后，分阶段判断完成程度

对应页面：After updating the files, verify what is actually complete

文件同步完成以后，也不能直接说功能已经做好了。还需要区分定义写完、实际运行通过，以及结果已经验证这几个阶段。

先检查角色规则、TXT 与 Registry 是否一致，再看工具权限与实际调用是否有运行证据，最后检查结果是否有依据并符合验收要求。上一页检查的是文字规则的一致性，这一页进一步问功能是否经过实际验证。

下面用项目里真实的修正做一个小例子。TODO 在 2026-10-15 明确，Active 代表定义完整并可被发现，不代表已经部署或完成运行验证。因此不能用一个状态字段替代这些不同层面的证据。

第 21 页｜人与 AI 的对话泳道图

对应页面：Human judgment and AI assistance at each iteration

讲图提示：进入页面先显示 Define。之后每按一次翻页，依次出现 Develop、Review、Record；每次连同对应的说明文字和箭头一起讲。

第一轮是 **Define，人到 AI**：我们提出目标、职责边界和要求，先把方向说清楚。

第二轮是 **Develop，AI 到人**：AI 整理想法、提出方案，生成供我们检查的文件。

第三轮是 **Review，人到 AI**：我们检查业务含义、冲突和依据，给出具体修改意见，而不是只说再优化一下。

第四轮是 **Record，AI 到人**：AI 把接受的决定、文件更新和下一步整理出来，交给我们核对。记录必须反映已接受的内容，不能把建议直接当成结论。

这张图回顾的是人与 AI 的协作，不是前面 Agent Hub 内部的执行泳道。人的判断贯穿每轮，AI 帮助整理、生成和记录，后续工作再从这些共同依据继续。

第 22 页｜五要素沟通模板：整理流程并建立 README

对应页面：A reusable communication template: design the workflow and README

讲图提示：先看左边五要素，再指右边同一段完整提示词。右侧五段组成一次请求，不是五次独立对话。

最后把前面的方法整理成一个听众可以直接复用的沟通模板。左边是五项：Goal 说明要实现什么，Context 交代依据，Scope 确定交付范围，Constraints 保留边界，Acceptance 定义怎样检查结果。

右边直接用搭建 Agent Hub 举例。我们先说明希望有一个统一入口来发现和协调业务 Agent，让 AI 阅读需求和已有文件，分清已定事项与待定问题。

然后明确要求它做什么：从用户可能的问法整理 Workflow，划分主 Agent、功能 Agent 和业务 Agent 的职责，提出最小文件结构，再搭建 README，包含愿景、架构、路由、定义规范、决策和待定问题。

同时要求推荐、规划和执行分开，执行前需要确认。没有确定的工具、部署和测试，不要自行写成已经完成。

最后的交付要能检查：流程图、文件职责和 README 草稿，再用几个代表性问题走一遍，把冲突、假设和下一步列出来。

如果项目已经有了，就先读当前 README 和流程图，再说明变更，最后同步相关文件。这个模板的作用，是让一次对话既有方向，也有依据、边界和可以审核的结果。

实际使用时多讲一点：这五项可以当作写请求前的检查表，不必每次都写很长。Goal 和 Scope 尤其要区分：Goal 是最后想解决什么，Scope 是本轮只交付什么。比如最终想搭建 Hub，本轮只需要角色表、主流程和 README 草稿，不要一下把全部 Agent 都生成出来。

Acceptance 要写成能检查的动作。比如让 AI 走三种请求：“只浏览”“指定 Agent 但缺输入”“需要多个 Agent”，逐个说明负责人、交接物和停止点。如果解释前后不一致，就指着冲突的步骤改，而不是只说“再优化一下”。这仍是设计走查，之后实际运行要有单独的验证记录。

也可以带回上次的 AE review：Goal 是给 deliverable review 提供可靠的 findings；Context 是当前数据、spec、RTF 和 log；Scope 是本轮核对 population、counts 和 percentages；Constraints 是按 distinct subject 统计、保留证据、不编造原因；Acceptance 是交付带当前值、重算值、来源和下一步的 issue table。这样听众就能看到，这个方法也能用于日常工作。

拿到第一版以后，先检查业务含义，再把接受的规则保存成 README 或模板，下一轮明确要求读取当前文件。本轮只要从一个任务、一组参照和一个可检查的交付开始，就已经可以用起来。

第 23 页｜结尾：设计、保存约定、持续核对

对应页面：From a conversation to a shared, reviewable design

讲图提示：从左到右收束三个动作，不再重复所有文件和角色名称。

最后想留下三个动作。第一，先设计逻辑，把角色、判断和交接说清楚。第二，保存共同参照，把已经接受的规则写进 README、流程图和 Agent 指令。第三，每次修改以后检查受影响的文件，并把定义完成和运行验证分开。

这样我们下一次和 AI 沟通时，就不必只靠重新解释，也能明确指出当前约定和希望修改的地方。当然，仍然需要要求 AI 读取这些参照，并对结果做审核。

卡片下面的三句话，是对整场内容的简短收束：把想法保存成背景，把要求保存成规范，用文件指导 AI 实现并接受检查。

从 Chat 到 Agent Hub，这次实践最值得复用的，就是把讨论逐步变成可以继续使用、也可以检查的项目上下文。谢谢大家。